

# HERO CYCLES PRICING ENGINE

## Part 1 – Problem Breakdown

### Understanding of the Problem

The given problem is about replacing the manual pricing process used by Hero Cycles. Currently, cycle pricing is managed using Excel sheets, where part prices are updated manually whenever costs change. This process is time-consuming and may lead to incorrect calculations.

The objective is to develop a web application that allows users to maintain cycle parts and cycle models while calculating the total price of a cycle automatically based on the latest part prices and selected quantities.

The application should be easy to use so that salespersons can quickly calculate the price of a cycle without performing manual calculations.

### Questions I Thought About Before Starting

While understanding the problem statement, I considered the following questions:

1. Should every cycle have a fixed set of parts or can different cycle models use different parts?
2. Should the application maintain previous prices or only the latest price?
3. Is GST or any discount calculation required?
4. Can the same part be used in multiple cycle models?
5. Should user authentication be implemented?

### Assumptions Made

To complete the assignment within the given time, I made the following assumptions:

- The application stores only the latest price of every part.
- Price history is not maintained.
- Taxes and discounts are not included in the price calculation.
- A salesperson manually enters the quantity of each required part.
- The application is intended for internal use.
- Only basic CRUD operations are required for cycle and part management.
- MySQL is used as the backend database.
- The application follows the Spring Boot MVC architecture.

# **Part 2 – Conceptual Solution**

## **Solution Approach**

I divided the application into three independent modules.

### **Module 1 – Part Management**

This module allows users to perform CRUD operations on cycle parts.

Features:

- Add Part
- View Parts
- Update Part
- Delete Part

Each part contains:

- Part ID
- Part Name
- Price

### **Module 2 – Cycle Management**

This module allows users to maintain available cycle models.

Features:

- Add Cycle
- View Cycle
- Update Cycle
- Delete Cycle

Each cycle contains:

- Cycle ID
- Cycle Name

### **Module 3 – Pricing Engine**

The Pricing Engine is responsible for calculating the total cycle price.

The user first selects a cycle model.

The application displays all available parts.

The salesperson enters the quantity required for each part.

The application calculates:

Part Total = Price  $\times$  Quantity

Finally,

Grand Total = Sum of all Part Totals

A price summary is displayed showing each part, its quantity, individual total and the overall cycle price.

## Pseudocode

START

Load all cycle models

Display cycle list

Read selectedCycle

Load all parts

grandTotal = 0

for each Part

    read quantity

    if(quantity > 0)

        partTotal = part.price \* quantity

        grandTotal = grandTotal + partTotal

    end if

end for

Display Price Summary

Display Grand Total

STOP

# Database Design

## Part Table:

|   | id   | name  | price |
|---|------|-------|-------|
| ▶ | 2    | tier  | 1000  |
|   | 6    | frame | 2500  |
|   | 7    | Break | 500   |
|   | 8    | seat  | 500   |
| * | NULL | NULL  | NULL  |

## Cycle Table

|   | id   | name          |
|---|------|---------------|
| ▶ | 2    | ghears        |
|   | 4    | kid           |
|   | 5    | Mountain bike |
|   | 8    | Road          |
| * | NULL | NULL          |

## Part 3 – Design and User Interface

The user interface was designed to be simple and easy to use.

The application provides separate pages for:

- Home Page
- Manage Parts
- Manage Cycles
- Pricing Engine
- Price Summary
- Error Page

The layout uses simple navigation and minimal user input so that the application is easy to understand even for first-time users.

## **AI Prompts Used During Development**

### **1. Understanding the Problem**

- Analyze the Hero Cycle Pricing Engine problem statement and explain the expected application flow.
- Suggest a simple design for a Hero Cycle Pricing Engine suitable for a Spring Boot MVC application.
- How can the pricing calculation be implemented efficiently in this project?

### **2. UI Design**

- Suggest a simple Thymeleaf UI for managing cycle parts and cycle models.
- Design a pricing page where the user selects a cycle and enters part quantities.
- Improve the layout and styling of the application's home page.

### **3. Pricing Logic**

- Suggest an approach for calculating the total cycle price based on selected quantities.
- How can the price summary be displayed using Thymeleaf?
- Design a simple pricing engine without adding unnecessary complexity.

### **4. Debugging**

- Help identify issues with Thymeleaf form binding.
- Explain why a controller mapping returns a 404 error.
- Help resolve issues related to CSS loading and page navigation.
- Explain the correct use of query parameters in Thymeleaf links.